

CTS SRE Day 7

Capacity Planning & Incident Management

EMPOWERING High Performance
Technology Teams

Agenda for the Day

Duration	Activity
10 mins	Welcome and Objectives
10 mins	Introduction to Capacity Planning & Incident Management
40 mins	Capacity Planning: Data-Driven Forecasting Use Case -1
10 mins	Break
30 mins	Incident Management & Response Protocols Use Case -2
30 mins	Automating Recovery & Self-Healing Mechanisms Use Case -3
30 mins	Continuous Improvement & Postmortems
10 mins	Summary & Q&A

Note: For sessions that are completely demo/Lab or notebook based, please keep one agenda slide and explain the same thing in the beginning so that participants are aware they do not have PPT.

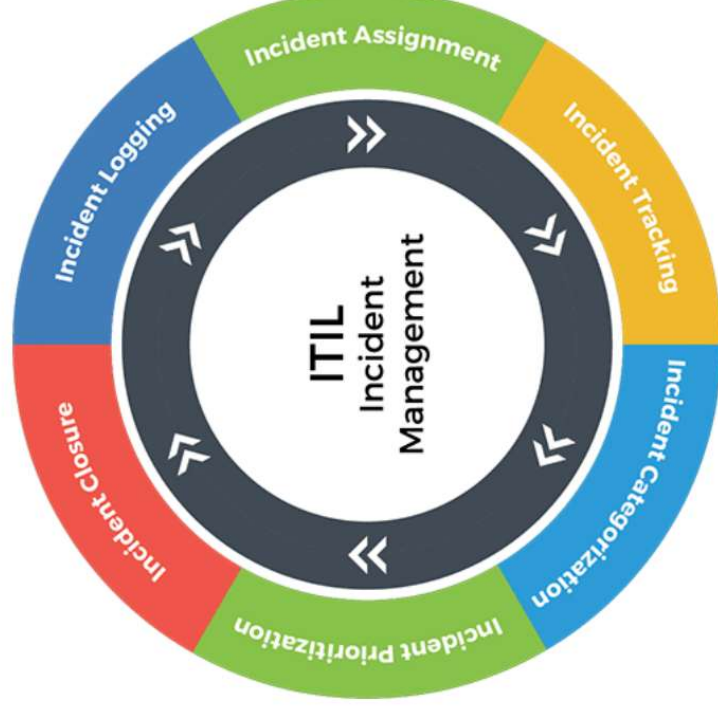
Introduction to Capacity Planning & Incident Management

- **Overview of Capacity Planning:** Capacity planning is the process of ensuring that IT infrastructure is scaled appropriately to meet both current and future demands. It involves forecasting resource requirements (e.g., CPU, memory, storage) and ensuring the system is designed to handle load spikes without performance degradation.
- **Key Goal:** Align infrastructure with business needs while maintaining optimal performance, avoiding bottlenecks, and minimizing resource wastage.

Why It Matters:

Ensures predictable scaling of resources
Reduces downtime with faster issue resolution
Balances cost optimization and performance
Enhances system reliability and availability

- **Overview of Incident Management:** Incident management is a process designed to restore normal service operations quickly in the event of an outage or failure. The goal is to minimize the impact on business and users.



Manage Your Capacity to Increase Your Availability

Maximize availability through proactive capacity management.

Business Challenges

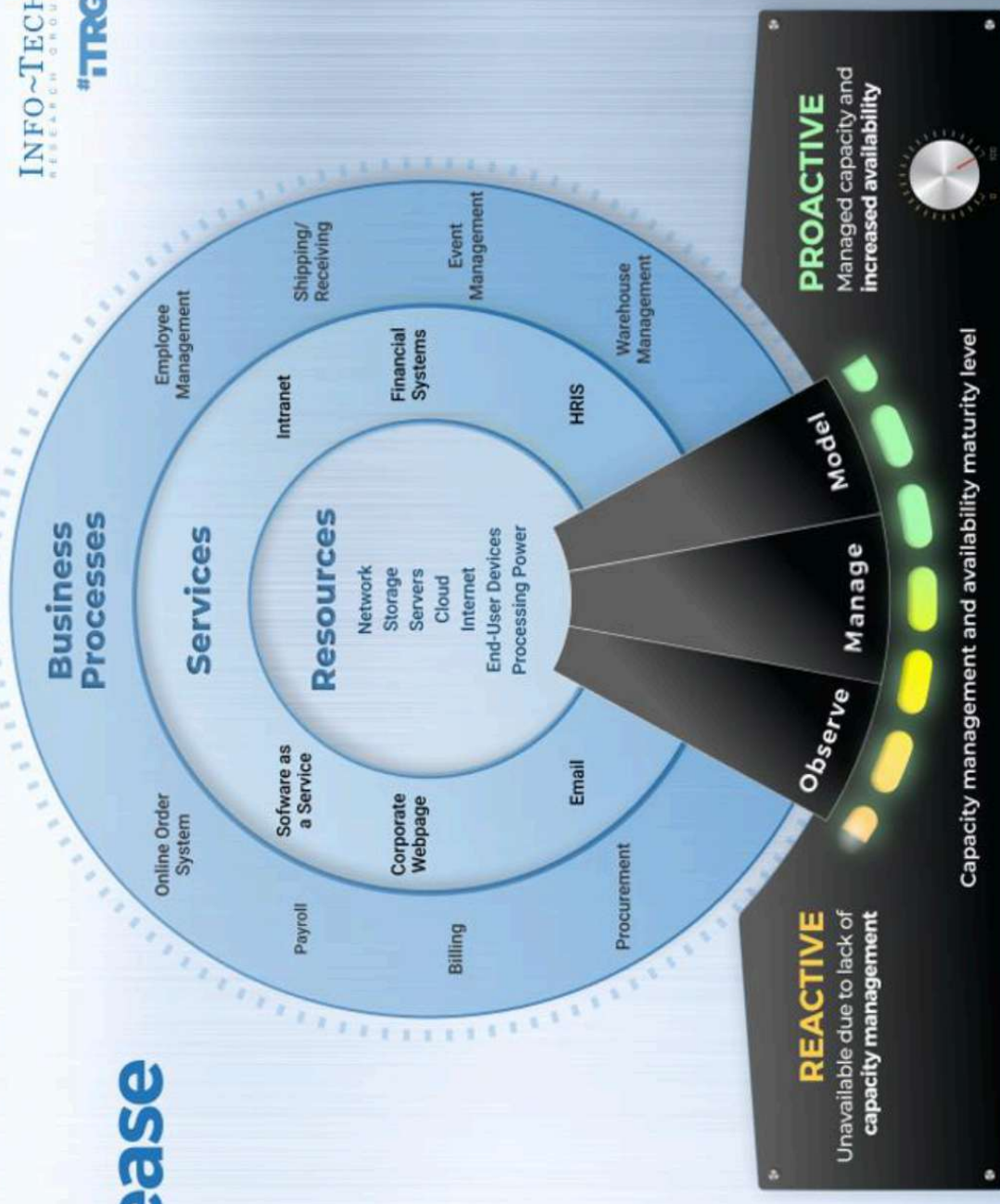
- ❖ Infrastructure is not aligned with current business initiatives.
- ❖ There is a lack of infrastructure resources for new initiatives.
- ❖ Existing capacity challenges lead to availability issues.

Capacity Management Plan

- ❖ **Observe** current components and business capacity requirements.
- ❖ **Manage** capacity adjustment solutions.
- ❖ **Model** future capacity requirements through proactive forecasting and adjustments.

Insight

The desire for better availability drives the evolution of monitoring into event management. This evolution enables better capacity management, which in turn improves availability.



Capacity Planning - Challenges & Best Practices

Common Challenges:

Over-provisioning leads to wasted resources
Under-provisioning causes performance degradation
Unpredictable demand spikes

Best Practices:

- ✓ Monitor usage trends (CPU, Memory, Storage, Network)
- ✓ Use auto-scaling for dynamic capacity adjustments
- ✓ Leverage predictive analytics for demand forecasting
- ✓ Simulate load testing before peak traffic



Case Study: Netflix – Scalable Capacity Planning for Peak Demand

Netflix, a global streaming platform, serves over 260 million subscribers. Their infrastructure must handle massive demand fluctuations, especially during peak hours, new show releases, or global events.

Challenges Faced

- **Traffic Spikes:** When popular shows (e.g., *Stranger Things* release) drop, traffic surges unpredictably.
- **Global Scalability:** Viewership varies across time zones, requiring adaptive scaling.
- **Cost Management:** Running idle resources for potential peaks would be expensive.

Best Practices Implemented

- ✓ Auto-Scaling & Predictive Analytics
- ✓ Chaos Engineering for Resilience
- ✓ Load Testing & Capacity Buffering
- ✓ Multi-Region Failover Strategy

Outcome

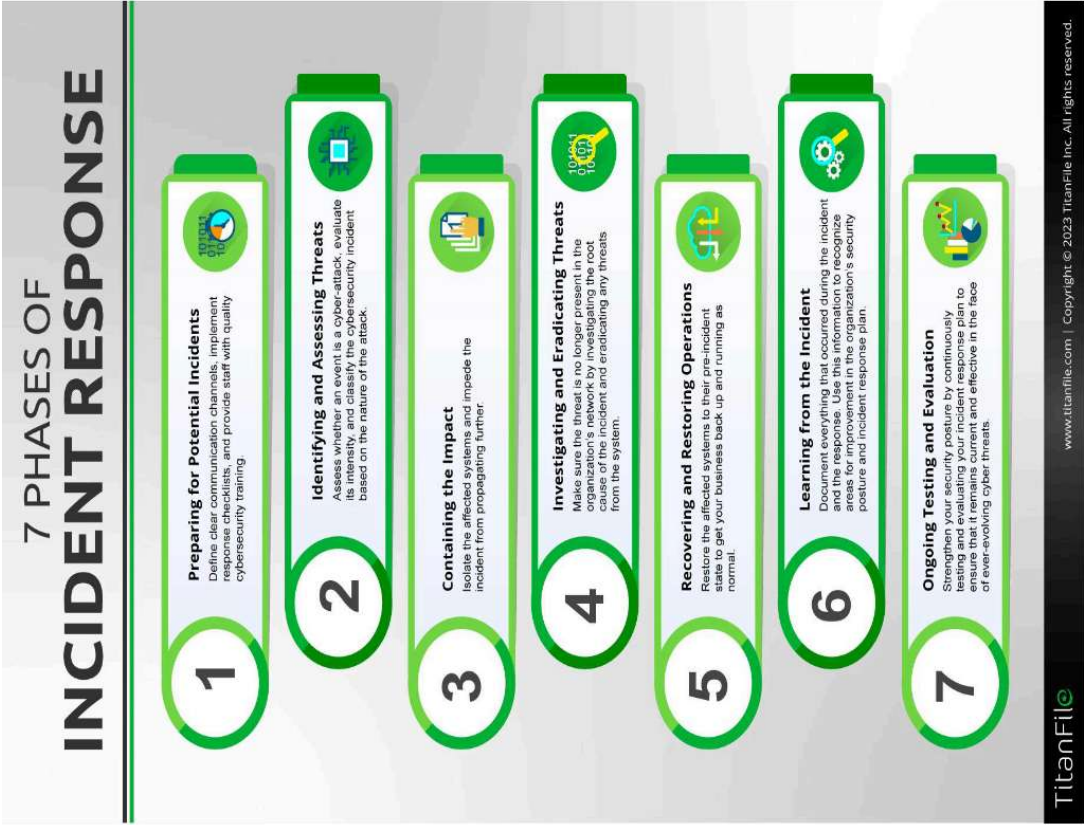
- 99.99% uptime even during peak releases
- Cost savings by avoiding unnecessary over-provisioning
- Seamless streaming experience with zero buffering during high-demand periods

Break

Incident Management - Defining the Process

Incident Lifecycle Stages:

- 1 **Detection** → Monitor logs & metrics in real-time
- 2 **Triage** → Categorize based on severity (Minor, Major, Critical)
- 3 **Escalation** → Notify on-call teams, follow pre-defined SOPs
- 4 **Resolution** → Mitigate the issue & apply a permanent fix
- 5 **Postmortem** → Document findings, refine future responses



Reducing MTTD & MTTR - Strategies & Tools

- ◆ **MTTD (Mean Time to Detect):** Time taken to detect a failure
- ◆ **MTTR (Mean Time to Resolve):** Time taken to fix an incident

Strategies to Reduce MTTD:

- ✓ Proactive monitoring with CloudWatch, Prometheus, Datadog
- ✓ Anomaly detection using AI-based alerting (Dynatrace, Splunk)
- ✓ Synthetic testing to detect failures before users do

Strategies to Reduce MTTR:

- ✓ Automate incident response playbooks
- ✓ Use real-time logs & tracing (ELK Stack, OpenTelemetry)
- ✓ Enable rollback strategies (Blue/Green Deployments etc)

Use Case – Payment Gateway Downtime

Scenario:

Issue: Users report failed payments during peak hours

Root Causes:

High API latency → Transaction timeouts
Database replication lag

Solution:

- 1 **Implement circuit breakers** to prevent cascading failures
- 2 **Set up synthetic transaction monitoring** (e.g., test payments every 5 minutes)
- 3 **Use auto-retry mechanisms** for failed transactions



Hands-On Activity: Simulate a **payment API failure** and observe alert triggers

Automating Recovery & Self-Healing

◆ Self-Healing Infrastructure

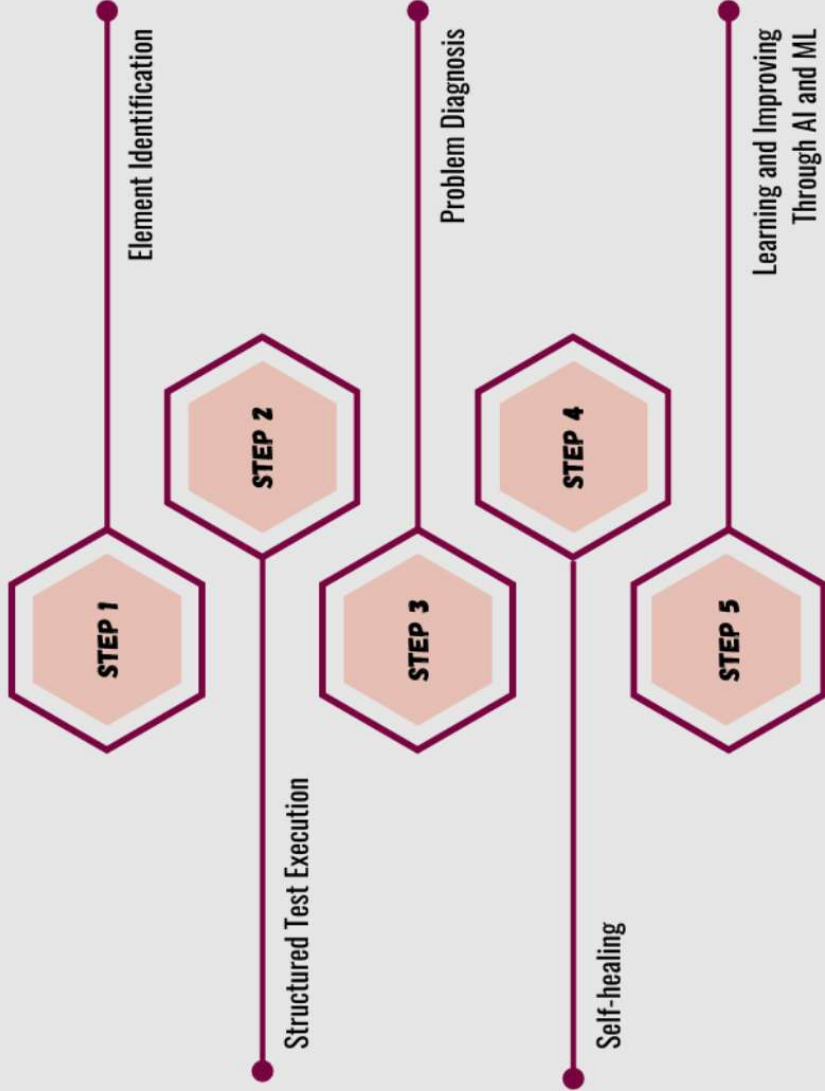
- ✓ Auto-restarting crashed services (Kubernetes Liveness Probes)
- ✓ Automatic failover for databases (Read Replicas, Multi-AZ RDS)

◆ Automating Operational Tasks

- ✓ Using AWS Lambda or Python Scripting for automated remediation.
- ✓ Triggering rollback on failed deployments

🔗 **Hands-On Activity:** Deploy **Kubernetes Liveness Probes** to auto-restart failing microservices

SELF-HEALING PROCESS



Use Case – Kubernetes Pod Failure & Self-Healing

Scenario:

Issue: Microservice crashes due to memory leak

Challenges:

How do we automatically restart failed pods?

How do we scale pods dynamically?

Solution:

- 1 **Configure Kubernetes Liveness & Readiness Probes**
- 2 **Enable Horizontal Pod Autoscaler (HPA) to scale based on CPU usage**
- 3 **Use AWS Auto-Recovery for EC2 instances**

 **Hands-On Activity:** Simulate pod failure & observe auto-restart behavior

Continuous Improvement Through Postmortems

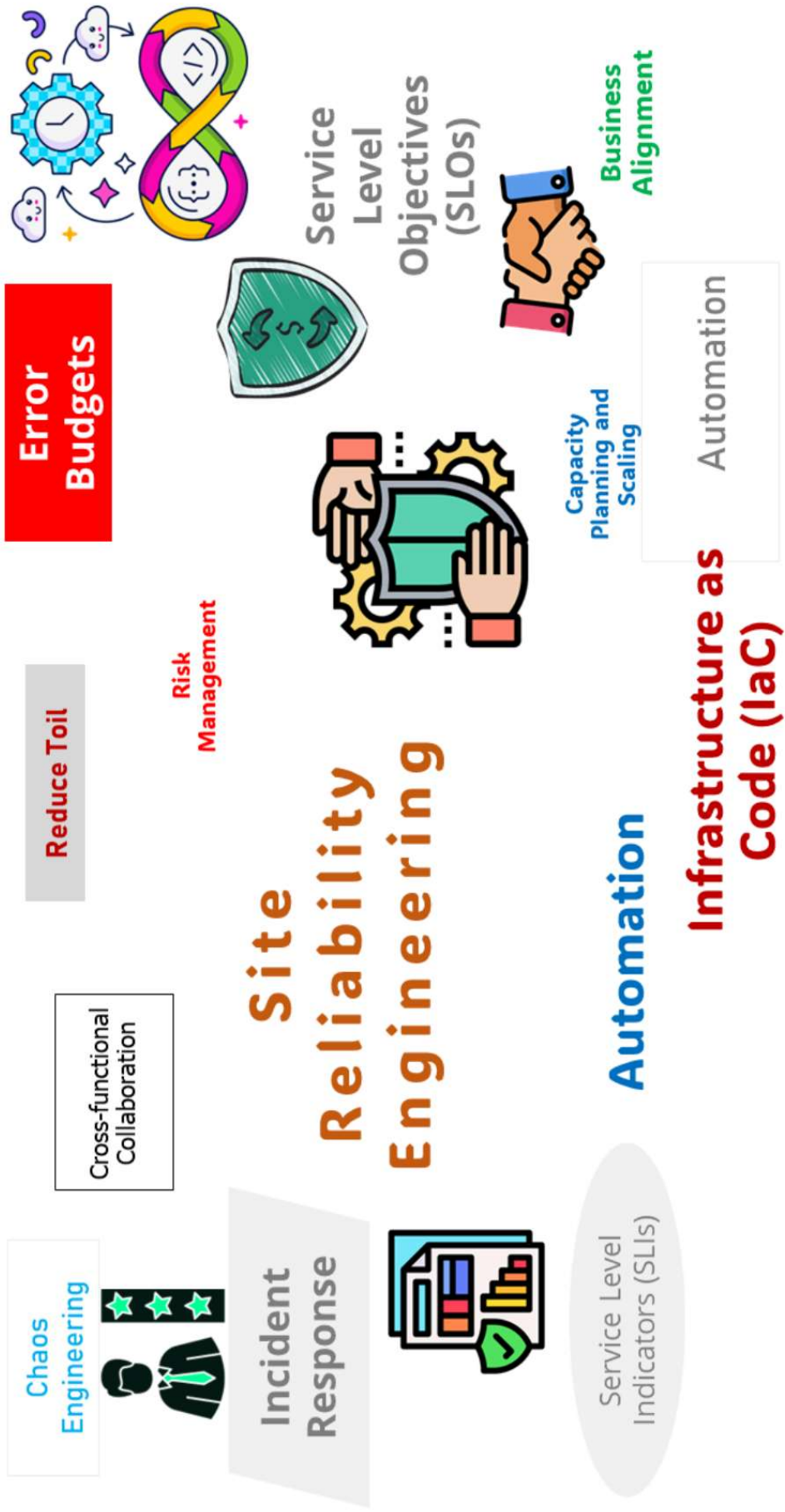
◆ Blameless Postmortems:

- ✓ Identify root cause (5 Whys, Fishbone Analysis)
- ✓ Document action items for future prevention
- ✓ Implement long-term fixes (e.g., automation, better alerting)

◆ Example Postmortem Template:

- 1 Incident Summary – What happened?
- 2 Root Cause Analysis – Why did it happen?
- 3 Impact Assessment – How many users affected?
- 4 Resolution & Fixes – How was it resolved?
- 5 Action Items – What needs improvement?

🔗 Hands-On Activity: Conduct a **mock postmortem** for a real-world AWS/GCP outage



Summary & Key Takeaways

- ✓ **Capacity planning ensures system stability & cost efficiency**
- ✓ **Incident management reduces downtime & accelerates recovery**
- ✓ **Automating recovery minimizes human intervention**
- ✓ **Self-healing systems improve long-term resilience**
- ✓ **Postmortems drive continuous learning & operational excellence**

Q&A & Discussion

Thank You!
StackRoute
(An NIIT Venture)

